

ONLINE SHOP PROJECT

Technical Documentation

1. Project Overview

Online Shop Project is an e-commerce web application built on a microservices architecture. Each service is responsible for a specific domain and can scale independently.

Key Features:

- Microservices architecture with independent scaling of each component
- Horizontal scaling: 3 replicas each for Auth, User, Payment, and Items services
- Nginx API Gateway with load balancing and failover
- Event-driven asynchronous communication between Items and Search services via RabbitMQ
- JWT authentication with session storage in Redis (Sentinel cluster)
- PostgreSQL with replication for Auth, User, and Payment databases
- MongoDB Replica Set for storing products and the search index
- Redis cache to speed up search queries
- React SPA frontend (Vite)

Repository	https://github.com/olenaazarova/online_shop_project
Backend language	Python 3 (FastAPI)
Frontend language	JavaScript (React + Vite)
Containerization	Docker Compose
Main port	8080 (API Gateway)

2. Architecture

2.1 Component Overview

The system consists of the following layers:

- Client layer: React SPA, Payment Frontend (static HTML)
- Gateway: Nginx API Gateway (port 8080) — routing and load balancing
- Service layer: Auth Service, User Service, Payment Service, Items Service, Search Service
- Event broker: RabbitMQ — items-events queue between Items and Search
- Data layer: PostgreSQL (x3 databases), MongoDB Replica Set, Redis Sentinel Cluster

2.2 Routing via API Gateway

Nginx forwards requests according to the following scheme:

URL Prefix	Upstream	Port
/api/auth/	auth-service-1/2/3	8000
/api/users/	user-service-1/2/3	8000
/api/payments/	payment-service-1/2/3	8008
/api/items/	items-service-1/2/3	8000
/api/search/	search-service	8000
/health	gateway (returns 200)	—

2.3 Failover and Load Balancing

Nginx is configured with active failover: max_fails=1, fail_timeout=5s. When one instance becomes unavailable, traffic is automatically redirected to the remaining instances. Connection timeout is 1s, read timeout is 5s.

3. Microservices

3.1 Authentication Service

Technologies	Python, FastAPI, PostgreSQL, Redis (Sentinel)
Replicas	3 (auth-service-1, auth-service-2, auth-service-3)
Database	auth_db (PostgreSQL, table: auth_users)
Cache / Sessions	Redis Sentinel Cluster (master: auth-redis)
JWT TTL	7200 seconds (2 hours)

Responsible for user registration and authentication. Stores credentials (email + password hash) in PostgreSQL. Sessions are stored in Redis with TTL. Passwords are hashed using PBKDF2-SHA256 with 120,000 iterations.

During registration the service atomically: (1) creates a record in the database, (2) calls User Service to create a profile. If User Service is unavailable, the database record is deleted to preserve transactional integrity.

API Endpoints

Method	Path	Auth	Description
POST	/api/auth/register	None	Register a new user. Returns user object + JWT token
POST	/api/auth/login	None	Login. Returns user object + JWT token
POST	/api/auth/logout	Bearer JWT	Logout. Deletes session from Redis
GET	/api/auth/me	Bearer JWT	Returns current authenticated user data
POST	/api/auth/validate	Bearer JWT	Validates token (used by other services)
GET	/health	None	Service health check

Table: auth_users

Field	Type	Description
id	UUID (PK)	Unique identifier (gen_random_uuid)
email	TEXT UNIQUE	User email (unique, stored lowercase)
password_hash	TEXT	PBKDF2-SHA256 password hash
created_at	TIMESTAMPTZ	Record creation timestamp
updated_at	TIMESTAMPTZ	Record update timestamp

Environment Variables

Variable	Default	Description
DATABASE_URL	""	PostgreSQL DSN connection string

REDIS_URL	redis://redis:6379/0	Redis URL (without Sentinel)
REDIS_SENTINELS	""	Sentinel hosts list (host:port, ...)
REDIS_MASTER_NAME	auth-redis	Redis master name in Sentinel
JWT_SECRET	local-development-secret	Secret key for signing JWT tokens
JWT_TTL_SECONDS	7200	JWT token lifetime in seconds
USER_SERVICE_URL	http://user-service:8000	URL of the User Service
INTERNAL_API_KEY	local-internal-key	Key for internal service-to-service calls

3.2 User Service

Technologies	Python, FastAPI, PostgreSQL
Replicas	3 (user-service-1, user-service-2, user-service-3)
Database	user_db (PostgreSQL, table: user_profiles)
Auth validation	JWT checked via Auth Service (/api/auth/validate)

Stores and manages user profiles. A profile is created automatically on registration via a protected internal endpoint (verified by X-Internal-API-Key header). Public endpoints are protected by JWT.

API Endpoints

Method	Path	Auth	Description
POST	/internal/users	X-Internal-API-Key	Internal: create user profile (called by Auth Service)
GET	/api/users/me	Bearer JWT	Get current user's profile
PATCH	/api/users/me	Bearer JWT	Update profile (first_name, last_name, phone)
GET	/health	None	Service health check

Table: user_profiles

Field	Type	Description
id	UUID (PK)	Unique profile identifier
auth_user_id	UUID	Foreign key to auth_users
email	TEXT	Email address (lowercase)
first_name	TEXT	First name
last_name	TEXT	Last name
phone	TEXT (nullable)	Phone number
role	TEXT	User role (default: user)
created_at / updated_at	TIMESTAMPTZ	Timestamps

3.3 Items Service

Technologies	Python, FastAPI, MongoDB (Motor async driver)
Replicas	3 (items-service-1, items-service-2, items-service-3)
Database	items_db (MongoDB Replica Set), collection: items
Message broker	RabbitMQ, queue: items-events

Manages the product catalog. After every create/update/delete operation, an event is published to RabbitMQ, which is consumed by Search Service to keep the search index up to date.

API Endpoints

Method	Path	Auth	Description
POST	/items/	None	Create a product. Publishes ITEM_CREATED event
GET	/items/	None	Retrieve all products
GET	/items/{item_id}	None	Retrieve a product by ID
PUT	/items/{item_id}	None	Update a product. Publishes ITEM_UPDATED event
DELETE	/items/{item_id}	None	Delete a product. Publishes ITEM_DELETED event

Item Document Schema (MongoDB)

Field	Type	Required	Description
id	string	Yes	MongoDB ObjectId converted to string
title	string	Yes	Product name
description	string	No	Product description
category	string	No	Product category
price	float	Yes	Price (must be ≥ 0)
quantity	int	Yes	Stock quantity (must be ≥ 0)

3.4 Search Service

Technologies	Python, FastAPI, MongoDB (PyMongo sync), Redis, RabbitMQ
Replicas	1
Database	items_db (MongoDB), collection: search_items
Cache	Redis (keys: search:*, TTL: 60 seconds)
Event consumer	RabbitMQ, queue: items-events

Provides product search by name with results cached in Redis. The search collection stays up to date by consuming events from Items Service in a background thread.

On receiving ITEM_CREATED/UPDATED/DELETED, the service updates the MongoDB search collection and invalidates the entire Redis cache (all search:* keys). Cache entries live for 60 seconds.

API Endpoints

Method	Path	Auth	Description
GET	/search/?q={query}	None	Search products by name (regex, case-insensitive). Results cached in Redis for 60s

RabbitMQ Events (items-events queue)

Event	Action in Search Service
ITEM_CREATED	insert_one into search_items + invalidate Redis cache
ITEM_UPDATED	update_one by id in search_items + invalidate Redis cache
ITEM_DELETED	delete_one by id in search_items + invalidate Redis cache

3.5 Payment Service

Technologies	Python, FastAPI, PostgreSQL
Replicas	3 (payment-service-1, payment-service-2, payment-service-3)
Database	payment_db (PostgreSQL)
Port	8008

Payment processing service. Accepts payment data as JSON or HTML form. Stores payments in PostgreSQL. Registers itself with the Config Server on startup for service discovery.

API Endpoints

Method	Path	Auth	Description
GET	/	None	Service health check
POST	/json/	None	Accept a payment (JSON body)
POST	/form_type/	None	Accept a payment (HTML form)

Payment Model

Field	Type	Description
user_id	int	User identifier
payment_id	int	Payment identifier
amount	float	Payment amount
item	string	Product name

Example Request (curl)

```
curl -v -H "Content-Type: application/json" \
  -d '{"user_id":10,"payment_id":11,"amount":5.0,"item":"laptop"}' \
  http://localhost:8080/api/payments/json/
```

3.6 Facade Service and Config Server

The Facade Service and Config Server are components of an early manual Service Discovery implementation. They are not used in the main `docker-compose.yml` (where Nginx handles routing), but are available via `compose.yaml`.

Config Server	FastAPI server (port 7999) for service registration and discovery
Facade Service	FastAPI server (port 8000) — aggregates calls to Payment Service
our_utils	Utility functions <code>register()</code> and <code>discover()</code> for communicating with Config Server

The Config Server address is configured in `conf.txt` in the project root.

4. Infrastructure

4.1 Databases

PostgreSQL (Bitnami PostgreSQL 16)

Three independent PostgreSQL clusters with master/slave replication:

Cluster	Master	Replicas	Used by
auth_db	auth-db-primary	auth-db-replica-1/2	Authentication Service
user_db	user-db-primary	user-db-replica-1/2	User Service
payment_db	payment-db-primary	payment-db-replica-1/2	Payment Service

MongoDB Replica Set (mongo:7)

Three nodes (items-mongo-1, items-mongo-2, items-mongo-3) in replica set items-rs. Initialized via a one-time items-mongo-init container. Used by both Items Service and Search Service.

Redis Sentinel Cluster

Topology: 1 master (redis-master) + 2 replicas (redis-replica-1/2) + 3 Sentinels (redis-sentinel-1/2/3). Quorum: 2 Sentinels. Failover timeout: 10 seconds.

4.2 RabbitMQ

Image	rabbitmq:3-management
AMQP port	5672
Management UI	15672 (default credentials: guest / guest)
Queue	items-events (auto_ack=True, durable=False)

4.3 Docker Compose Networking

All services run in a single default Docker Compose network. Container names are used as DNS hostnames (auth-service-1, redis-master, etc.). Externally exposed ports: 8080 (Nginx), 5672 and 15672 (RabbitMQ).

5. Frontend

Technologies	React 18, Vite, React Router DOM
Location	/frontend
Routes	/, /cart, /profile, /register

SPA Routes

Route	Component / Purpose
/	Home — main product catalog page
/cart	Cart — shopping cart
/profile	Profile — user profile page
/register	Register — registration form

Payment Frontend

A separate static HTML file (payment-frontend/index.html) for demonstrating the payment form. Not integrated into Nginx — used directly or during development.

Running in Development

```
cd frontend
npm install
npm run dev
```

The Vite dev server starts on port 5173 by default. API requests are proxied via vite.config.js.

6. Deployment

6.1 Requirements

- Docker Engine 24+ and Docker Compose v2
- Git
- Minimum 4 GB RAM (8 GB recommended for all replicas)

6.2 Starting the Application

Clone the repository and start all services with a single command:

```
git clone https://github.com/olenaazarova/online_shop_project.git
cd online_shop_project
docker-compose up --build
```

Once started:

- API Gateway available at <http://localhost:8080>
- RabbitMQ Management UI at <http://localhost:15672> (guest / guest)

6.3 Verification

Gateway health check

```
curl http://localhost:8080/health
```

Expected response: gateway ok

User registration

```
curl -X POST http://localhost:8080/api/auth/register \
-H "Content-Type: application/json" \
-d '{"email":"user@example.com","password":"secret123",
    "first_name":"John","last_name":"Doe"}'
```

Search for products

```
curl http://localhost:8080/api/search/?q=laptop
```

Create a product

```
curl -X POST http://localhost:8080/api/items/items/ \
-H "Content-Type: application/json" \
-d '{"title":"Laptop Pro","price":999.99,"quantity":10}'
```

6.4 Production Recommendations

- Replace JWT_SECRET, INTERNAL_API_KEY, and database passwords with strong, random values
- Configure HTTPS at the Nginx level (TLS certificates)
- Use an external Redis Sentinel or Redis Cluster
- Consider Kubernetes for orchestration instead of Docker Compose
- Add authorization to Items Service endpoints
- Configure RabbitMQ with durable queues and persistent messages

